Orchid Project

OVERVIEW OF WHAT I NEED TO BUILD

I will create a full-stack web app that:

1. Takes a public website URL as input (via a UI).
2. Scrapes that website to extract its **design context** (HTML structure, styles, etc.).
3. Sends that info to an **LLM (like GPT-4 or Claude)** via your FastAPI backend.
4. LLM returns a simplified, visually similar **HTML version**.
5. The frontend displays cloned HTML as a preview.

This is a full-stack web app built as part of the SWE internship take-home project for Orchids. It allows users to input a public website URL and see a cloned version generated using a scraping + LLM pipeline (with a fallback system if OpenAI API quota is exceeded).

File Structure:

orchids-challenge/

├── README.md

├── frontend/ ← Next.js + TypeScript UI

└── backend/ (likely) ← FastAPI Python backend (you'll build or connect)

Features

- Frontend in **Next.js (TypeScript)**: Takes a website URL and renders the cloned output

- Backend in **FastAPI (Python)**: Scrapes the input URL and builds a design prompt

- Integrated with **OpenAI GPT-4o** to generate visually similar HTML

- **Fallback mock HTML** used if API quota is exceeded, ensuring full functionality

Tech Stack

Layer	Tech
Frontend	Next.js, TypeScript, Tailwind-inspired styling
Backend	FastAPI, Python 3, BeautifulSoup, requests
AI/LLM	OpenAI (GPT-4o via `openai` SDK)
Extras	Graceful fallback mechanism, error handling

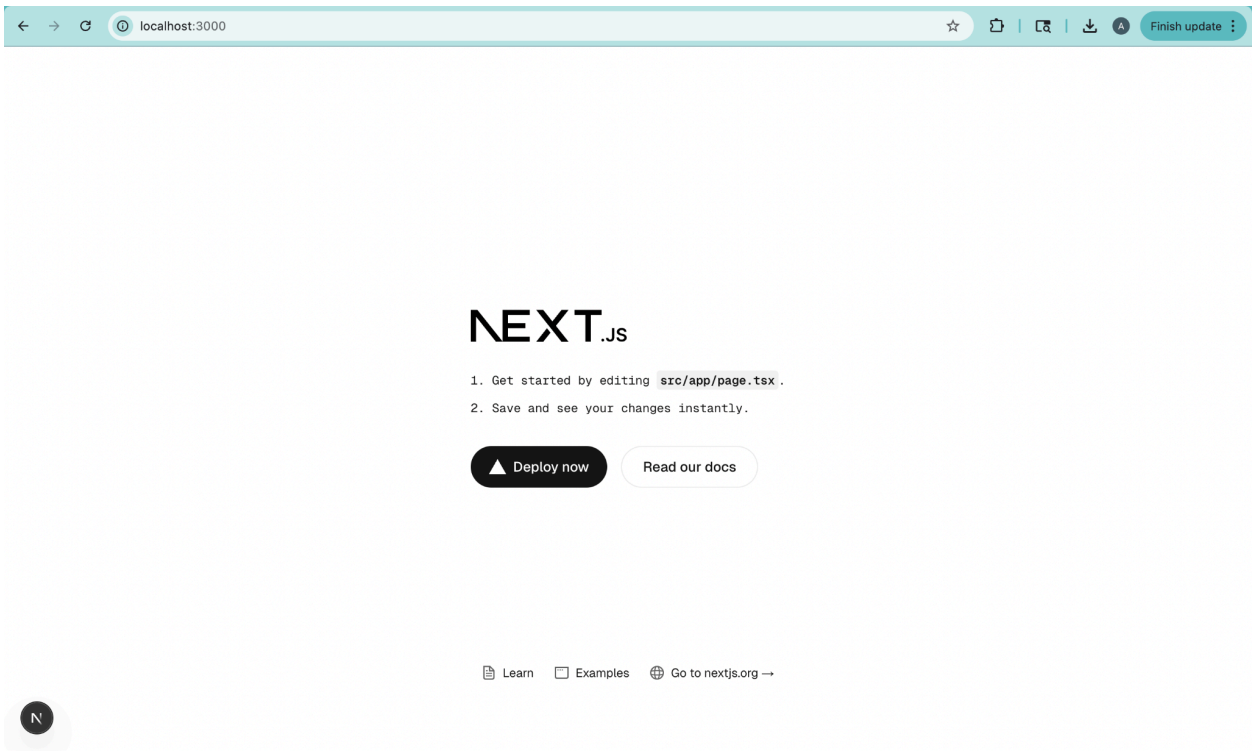
STEP 1: SETUP THE PROJECT LOCALLY

- **Install frontend dependencies and run it**

cd orchids-challenge/frontend

npm install

npm run dev



STEP 2: SET UP MAIN.PY INSIDE /BACKEND with website scraping

Change - orchids-challenge/backend/main.py

```
    from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
import requests
from bs4 import BeautifulSoup
from openai import OpenAI
import re

# Create FastAPI app
app = FastAPI()

# Set up OpenAI client with correct v1.x syntax
client =
OpenAI(api_key="sk-proj-fORwAkMJcbKENu0rKPVfF2CTkRC7bA-CryIwgdMisw8J1XxBI6c62nLoGmKagK
NYnhVbbtzOIjT3BlbkFJJHg0kJ4SYhtq0HHRbKvwcJ-LsYbBbb5UutmU_nyNT_SmSf2G5uAEOMpxwvaDbJrNka
Vxd4i-cA")

# CORS middleware
app.add_middleware(
   CORSMiddleware,
   allow_origins=["*"],
   allow_methods=["*"],
   allow_headers=["*"],
)

# Request body schema
class CloneRequest(BaseModel):
   url: str

# Clean LLM output
```

```python
def clean_llm_output(raw: str) -> str:
    cleaned = re.sub(r"```html|```", "", raw)
    return cleaned.strip()

# Fallback HTML generator
def fallback_html(prompt: str) -> str:
    return f"""
    <html>
      <head>
        <title>Mock AI Clone</title>
        <style>
          body {{ font-family: Arial; background: #f0f0f0; padding: 2rem; }}
          pre {{ background: #eee; padding: 1rem; }}
        </style>
      </head>
      <body>
        <h1>Mock AI-generated clone</h1>
        <p>Note: OpenAI API call failed or quota exceeded. Using fallback output.</p>
        <h2>🧠 Prompt Sent:</h2>
        <pre>{prompt[:1000]}</pre>
      </body>
    </html>
    """

# Generate HTML via LLM with fallback logic
def generate_html_with_llm(prompt: str) -> str:
    try:
        response = client.chat.completions.create(
            model="gpt-4o",
            messages=[
                {"role": "system", "content": "You are an expert web designer."},
                {"role": "user", "content": prompt}
            ]
        )
        llm_output = response.choices[0].message.content
        return clean_llm_output(llm_output)
    except Exception as e:
        print("⚠️ LLM call failed. Using fallback. Reason:", str(e))
        return fallback_html(prompt)

# Main cloning endpoint
@app.post("/clone")
```

```python
async def clone_website(data: CloneRequest):
    try:
        response = requests.get(data.url)
        soup = BeautifulSoup(response.text, 'html.parser')

        title = soup.title.string if soup.title else "No Title"
        body = soup.body.prettify() if soup.body else "No Body"

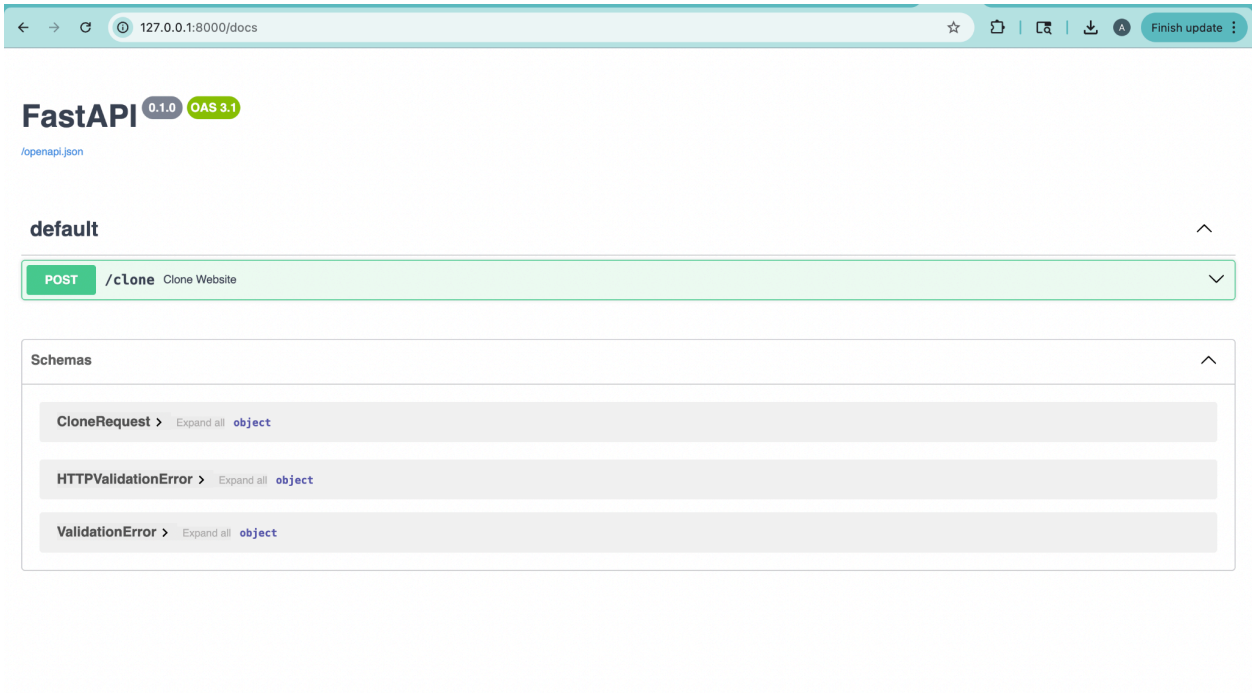
        prompt = (
            f"Generate HTML for a webpage that looks visually similar to a site titled '{title}'.\n\n"
            f"The page body contains:\n{body[:2000]}\n\n"
            "Generate a simple but styled HTML page using semantic tags, with readable structure and placeholder content."
        )

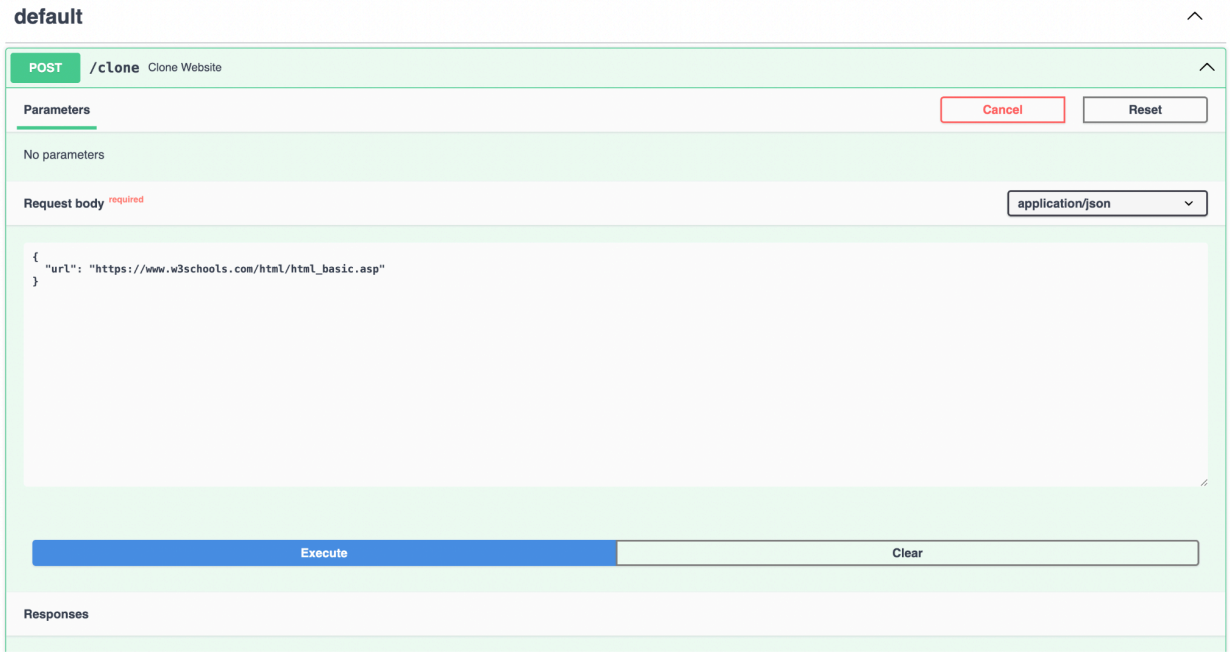
        html = generate_html_with_llm(prompt)
        return {"html": html}

    except Exception as e:
        print("ERROR:", str(e))
        return {"html": f"<h1>Internal Error: {str(e)}</h1>"}
```

Bash:

```
cd backend
uvicorn main:app --reload --port 8000
```


127.0.0.1:8000/docs
Finish update
FastAPI
0.1.0
OAS 3.1
/openapi.json
default
POST
/clone
Clone Website
Schemas
CloneRequest
Expand all
object
HTTPValidationError
Expand all
object
ValidationError
Expand all
object


default
POST
/clone
Clone Website
Parameters
Cancel
Reset
No parameters
Request body
required
application/json
{
"url": "https://www.w3schools.com/html/html_basic.asp"
}
Execute
Clear
Responses

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/clone' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "url": "https://www.w3schools.com/html/html_basic.asp"
}'
```

Request URL

```
http://127.0.0.1:8000/clone
```

Server response

Code	Details
200	**Response body**


```
{
  "html": "\n        <html>\n        <head>\n            <title>HTML Basic</title>\n            <style>\n                body { font-family: Arial, sans-serif; padding: 20px; background: #f9f9f9; }\n                h1 { color: #333; }\n            </style>\n        </head>\n        <body>\n            <h1>Cloned from: https://www.w3schools.com/html/html_basic.asp</h1>\n            <body>\n <div id=\"tnb-search-suggestions\">\n </div>\n <div class=\"classic\" id=\"top-nav-bar\">\n  <div class=\"w3-bar notranslate w3-white\" id=\"pagetop\">\n   <a aria-label=\"Home link\" class=\"w3-bar-item w3-button w3-hover-none w3-left ga-top ga-top-w3home\" href=\"https://www.w3schools.com\" id=\"w3-logo\" style=\"width: 75px\" title=\"Home\">\n    <i aria-hidden=\"true\" class=\"fa fa-logo ws-hover-text-green\" style=\"\n          position: relative;\n          z-index: 1;\n color: #04aa6d;\n          font-size: 36px !important;\n        \">\n    </i>\n   </a>\n   <nav class=\"tnb-desktop-nav w3-bar-item\">\n    <a class=\"tnb-nav-btn w3-bar-item w3-button barex bar-item-hover w3-padding-16 ga-top ga-top-tut-and-ref\" href=\"javascript:void(0)\" id=\"navbtn_tutorials\" onclick=\"TopNavBar.openNavItem('tutorials')\" role=\"button\" title=\"Tutorials and References\">\n     Tutorials\n     <i aria-hidden=\"true\" class=\"fa fa-caret-down\" style=\"font-size: 15px\">\n     </i>\n     <i aria-hidden=\"true\" class=\"fa fa-caret-up\" style=\"display: none; font-size: 15px\">\n     </i>\n    </a>\n    <a class=\"tnb-nav-btn w3-bar-item w3-button barex bar-item-hover w3-padding-16 ga-top ga-top-exc-and-quz\" href=\"javascript:void(0)\" id=\"navbtn_exercises\" onclick=\"TopNavBar.openNavItem('exercises')\" role=\"button\" title=\"Exercises and Quizzes\">\n     Exercises\n     <i aria-hidden=\"true\" class=\"fa fa-caret-down\" style=\"font-size: 15px\">\n     </i>\n     <i aria-hidden=\"true\" class=\"fa fa-caret-up\" style=\"display: none; font-size: 15px\">\n     </i>\n    </a>\n    <a class=\"tnb-nav-btn w3-bar-item w3-button barex bar-item-hover w3-padding-16 tnb-paid-service ga-to
```



```
server: uvicorn
```


Responses

Code	Description	Links
200	Successful Response Media type: application/json Controls Accept header. **Example Value** | Schema `"string"`	*No links*
422	Validation Error Media type: application/json **Example Value** | Schema	*No links*


```
{
  "detail": [
    {
      "loc": [
        "string",
        0
```

STEP 3: CONNECT FRONTEND TO BACKEND

In the frontend code (likely in `page.tsx` or similar):

```
'use client';

import { useState } from 'react';

export default function Home() {

 const [inputUrl, setInputUrl] = useState('');

 const [clonedHTML, setClonedHTML] = useState('');

 const [loading, setLoading] = useState(false);

 const [error, setError] = useState('');



 const fetchClonedHTML = async () => {

   setLoading(true);

   setError('');

   try {

     const res = await fetch('http://localhost:8000/clone', {

       method: 'POST',

       headers: {

         'Content-Type': 'application/json',

       },

       body: JSON.stringify({ url: inputUrl }),

     });
```

```
    if (!res.ok) {
      throw new Error('Failed to clone website');
    }

    const data = await res.json();
    setClonedHTML(data.html);
  } catch (err: any) {
    setError(err.message || 'Something went wrong');
  } finally {
    setLoading(false);
  }
};

return (
  <main className="min-h-screen p-10 bg-gray-100">
    <h1 className="text-3xl font-bold mb-6">🌐 Website Cloner</h1>

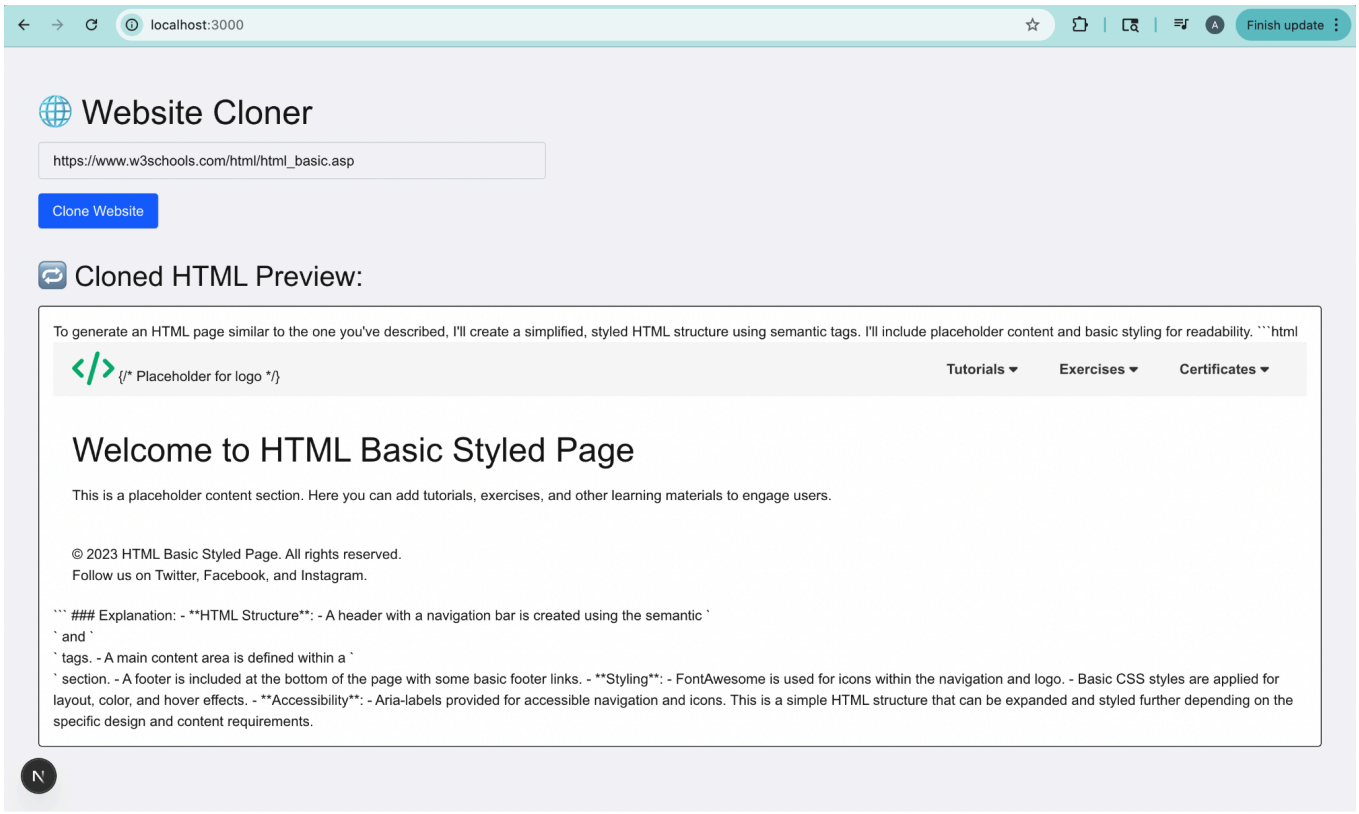
    <div className="max-w-xl mb-4">
      <input
        type="text"
        value={inputUrl}
        onChange={(e) => setInputUrl(e.target.value)}
        placeholder="Enter a public website URL (e.g., https://example.com)"
        className="border border-gray-300 rounded px-4 py-2 w-full"
```

```
        />
      </div>

      <button
        onClick={fetchClonedHTML}
        className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700"
        disabled={!inputUrl || loading}
      >
        {loading ? 'Cloning...' : 'Clone Website'}
      </button>

      {error && <p className="text-red-600 mt-4">{error}</p>}

      <div className="mt-8">
        <h2 className="text-xl font-semibold mb-2">🔁 Cloned HTML Preview:</h2>
        <div
          className="bg-white border rounded p-4"
          dangerouslySetInnerHTML={{ __html: clonedHTML }}
        />
      </div>
    </main>
  );
}
```


localhost:3000
Finish update
Website Cloner
https://www.w3schools.com/html/html_basic.asp
Clone Website
Cloned HTML Preview:
To generate an HTML page similar to the one you've described, I'll create a simplified, styled HTML structure using semantic tags. I'll include placeholder content and basic styling for readability. ```html
{/* Placeholder for logo */}
Tutorials
Exercises
Certificates
Welcome to HTML Basic Styled Page
This is a placeholder content section. Here you can add tutorials, exercises, and other learning materials to engage users.
© 2023 HTML Basic Styled Page. All rights reserved.
Follow us on Twitter, Facebook, and Instagram.
``` ### Explanation: - **HTML Structure**: - A header with a navigation bar is created using the semantic `
` and `
` tags. - A main content area is defined within a `
` section. - A footer is included at the bottom of the page with some basic footer links. - **Styling**: - FontAwesome is used for icons within the navigation and logo. - Basic CSS styles are applied for layout, color, and hover effects. - **Accessibility**: - Aria-labels provided for accessible navigation and icons. This is a simple HTML structure that can be expanded and styled further depending on the specific design and content requirements.
```